GloVe: Global Vectors for Word Representation

Architecture & Internals of the Stanford Reference Implementation

Companion document for the `stanfordnlp/GloVe` source tree

October 13, 2026

Abstract

This document explains GloVe (Global Vectors for Word Representation) end to end, from the mathematical model to the bytes on disk. Part I covers the *why*: the idea that ratios of co-occurrence probabilities encode meaning, and how that idea becomes a weighted least-squares objective. Part II covers the *how*: the four-stage C pipeline (`vocab_count` → `cooccur` → `shuffle` → `glove`), the data structures and file formats that connect the stages, the memory-management heuristics, and the lock-free multi-threaded AdaGrad (adaptive gradient) trainer. Part III covers operations: hyper-parameters, resource sizing, evaluation, reproducibility, and known quirks found while reading the code. Each section points to the functions in `src/` that implement it, so you can read the document and the code side by side.

Contents

I The Model **3**

1 The problem GloVe solves **3**

2 Notation and abbreviations **3**

3 From probability ratios to a loss function **4**
3.1 The core observation . . . 4
3.2 Deriving the model . . . 4
3.3 The weighted least-squares objective . . . 5
3.4 Gradients . . . 6
3.5 Relationship to other models . . . 6

II The Implementation **6**

4 Repository map **6**

5 The pipeline at a glance **6**

6 Shared foundations: `common.c` **7**
6.1 The universal record: `CREC` . . . 7
6.2 Tokenisation contract: `get_word()` . . . 8
6.3 Hash table . . . 8

7 Stage 1 — `vocab_count` **8**

8 Stage 2 — `cooccur` 8
8.1 Windowed counting . . . 9
8.2 Hybrid dense/sparse storage . . . 9
8.3 External k-way merge . . . 10
8.4 Choosing `max_product` from `-memory` . . . 10

9 Stage 3 — `shuffle` 11

10 Stage 4 — `glove`, the trainer 11
10.1 Parameter memory layout . . . 12
10.2 The inner loop . . . 12
10.3 Hogwild parallelism . . . 13
10.4 Outputs . . . 13

III Operating GloVe 14

11 Hyper-parameter reference 15

12 Resource sizing 15

13 Evaluation 16

14 Reproducibility 16

15 Known quirks found while reading the code 16

16 Mental model: one-paragraph summary 17

References 17

Part I
The Model

1 The problem GloVe solves

A *word embedding* maps each vocabulary word i to a dense vector $\mathbf{w}_i \in \mathbb{R}^d$ (typically $d \in \{50, 100, 200, 300\}$) so that geometric relationships between vectors mirror semantic and syntactic relationships between words. The classic demonstration is analogy arithmetic:

$$\mathbf{w}_{\text{king}} - \mathbf{w}_{\text{man}} + \mathbf{w}_{\text{woman}} \approx \mathbf{w}_{\text{queen}}.$$

Before GloVe there were two main families of methods:

Global matrix factorisation Latent Semantic Analysis (LSA), Hyperspace Analogue to Language (HAL), and positive pointwise mutual information (PPMI) followed by singular value decomposition (SVD). Build a large word–context count matrix over the whole corpus and factorise it. These use corpus-wide statistics efficiently, but they do poorly on analogy tasks because raw or log counts are dominated by frequent words and SVD weights every cell the same.

Local context-window prediction (word2vec's skip-gram and continuous bag-of-words (CBOW) models). Slide a window over the text and train a classifier to predict context words. These produce good linear structure, but they scan the corpus window by window and never use the aggregate counts directly, so repeated co-occurrences are processed again each time they appear.

GloVe (Pennington, Socher & Manning, Conference on Empirical Methods in Natural Language Processing (EMNLP) 2014) combines the two. It **collects global co-occurrence counts once**, then **fits a log-bilinear regression model** to the non-zero counts. This gives word2vec-style linear structure at the cost of *factorising a matrix*: training scales with the number of non-zero entries of the co-occurrence matrix, not with corpus length.

2 Notation and abbreviations

Symbol	Meaning
V	Vocabulary size (number of lines in `vocab.txt`).
d	Vector dimension (`-vector-size`).
$X \in \mathbb{R}^{V \times V}$	Co-occurrence matrix. X_{ij} is the (distance-weighted) number of times word j appears in the context of word i.
$X_i = \sum_k X_{ik}$	Total context mass of word i.
$P_{ij} = X_{ij}/X_i$	Probability that j appears in the context of i.
$\mathbf{w}_i,\ \tilde{\mathbf{w}}_j \in \mathbb{R}^d$	*Word* vector of i and *context* vector of j (two separate parameter sets).
$b_i,\ \tilde{b}_j \in \mathbb{R}$	Word and context biases.
$f(\cdot)$	Weighting function, with parameters $x_{\max}$ (`-x-max`) and α (`-alpha`).
$\text{nnz}(X)$	Number of non-zero entries of X.

Abbreviations

Each abbreviation is also spelled out where it first appears in the text.

AdaGrad	Adaptive gradient (optimiser)	NaN	Not a number
CBOW	Continuous bag-of-words (word2vec model)	NeurIPS	Neural Information Processing Systems (conference)
CREC	Co-occurrence record (C struct)	nnz	Number of non-zero entries
EMNLP	Empirical Methods in Natural Language Processing (conference)	PMI	Pointwise mutual information
EOF	End of file	PPMI	Positive pointwise mutual information
GB	Gigabyte (2^{30} bytes in the code)	pthreads	POSIX threads
GloVe	Global Vectors for Word Representation	PTB	Penn Treebank
HAL	Hyperspace Analogue to Language	RAM	Random-access memory
HASHREC	Hash-table record (C struct)	SGD	Stochastic gradient descent
I/O	Input/output	SSD	Solid-state drive
JMLR	Journal of Machine Learning Research	stdin, stdout, stderr	Standard input, output, error
LSA	Latent Semantic Analysis	SVD	Singular value decomposition
TSIZE	Hash-table size (constant)	UTF-8	Unicode Transformation Format, 8-bit

3 From probability ratios to a loss function

3.1 The core observation

Take two related words, $i = \text{ice}$ and $j = \text{steam}$, and ask how probe words k co-occur with each of them. The paper reports (6-billion-token corpus):

Probe k	solid	gas	water	fashion
$P(k \mid \text{ice})$	1.9×10^{-4}	6.6×10^{-5}	3.0×10^{-3}	1.7×10^{-5}
$P(k \mid \text{steam})$	2.2×10^{-5}	7.8×10^{-4}	2.2×10^{-3}	1.8×10^{-5}
Ratio	**8.9**	$\mathbf{8.5 \times 10^{-2}}$	1.36	0.96

The raw probabilities are hard to read. The *ratio* P_{ik}/P_{jk} is clear: it is large for words specific to ice (*solid*), small for words specific to steam (*gas*), and close to 1 for words related to both (*water*) or to neither (*fashion*). **Meaning is carried by ratios of co-occurrence probabilities.**

3.2 Deriving the model

We want a function F with

$$F(\mathbf{w}_i, \mathbf{w}_j, \tilde{\mathbf{w}}_k) = \frac{P_{ik}}{P_{jk}}. \tag{1}$$

Three requirements narrow it down:

1. **Vector spaces are linear**, so we compare i and j through their difference: $F(\mathbf{w}_i - \mathbf{w}_j, \tilde{\mathbf{w}}_k)$.
2. **The right-hand side is a scalar**, so we reduce the arguments with a dot product: $F((\mathbf{w}_i - \mathbf{w}_j)^\top \tilde{\mathbf{w}}_k)$. This keeps the dimensions from mixing in arbitrary ways.
3. **Word and context roles are interchangeable** (X is symmetric for a symmetric window), so F should turn differences into ratios. That makes it a homomorphism from $(\mathbb{R}, +)$ to

$(\mathbb{R}_{>0}, \times)$:

$$F((\mathbf{w}_i - \mathbf{w}_j)^\top \tilde{\mathbf{w}}_k) = \frac{F(\mathbf{w}_i^\top \tilde{\mathbf{w}}_k)}{F(\mathbf{w}_j^\top \tilde{\mathbf{w}}_k)},$$

whose solution is $F = \exp$.

Hence $\exp(\mathbf{w}_i^\top \tilde{\mathbf{w}}_k) = P_{ik} = X_{ik}/X_i$, i.e.

$$\mathbf{w}_i^\top \tilde{\mathbf{w}}_k = \log X_{ik} - \log X_i.$$

The term $\log X_i$ does not depend on k, so it is absorbed into a bias b_i. A context bias $\tilde{b}_k$ restores the symmetry:

$$\boxed{\mathbf{w}_i^\top \tilde{\mathbf{w}}_k + b_i + \tilde{b}_k \;=\; \log X_{ik}} \tag{2}$$

3.3 The weighted least-squares objective

Equation (2) cannot hold exactly, so we fit it by least squares. Two problems remain: $\log 0$ is undefined, and rare co-occurrences are noisy while very frequent ones (*the*, *of*) should not dominate. GloVe adds a weight $f(X_{ij})$ to each term:

$$J \;=\; \sum_{i,j\,:\,X_{ij}>0} f(X_{ij}) \left(\mathbf{w}_i^\top \tilde{\mathbf{w}}_j + b_i + \tilde{b}_j - \log X_{ij}\right)^2 \tag{3}$$

$$f(x) = \begin{cases} (x/x_{\max})^\alpha & x < x_{\max} \\ 1 & \text{otherwise} \end{cases} \tag{4}$$

The sum runs only over **non-zero** cells. That is why the pipeline only ever stores sparse (i, j, X_{ij}) triples. It also requires $f(0) = 0$, so zero cells contribute nothing.

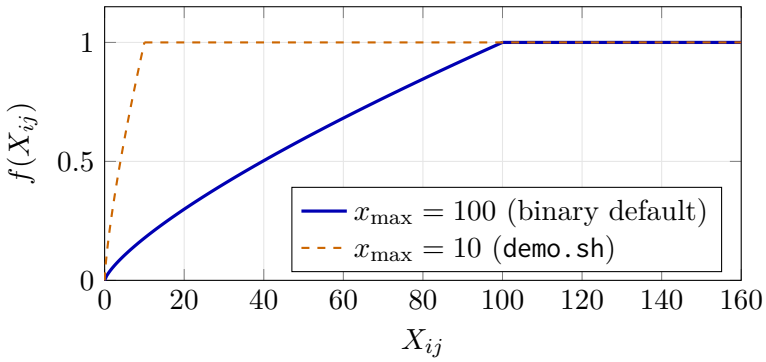


Figure 1: The weighting function with $\alpha = 0.75$. Rare pairs are down-weighted and frequent pairs are capped at weight 1.

Key insight

- $\alpha = 3/4$ is the same exponent word2vec uses for its negative-sampling distribution. Empirically it beats $\alpha = 1$.
- $x_{\max}$ should match corpus scale. The binary's default is 100 (the paper's value for multi-billion-token corpora). `demo.sh` and `Training_README.md` use 10.
- GloVe learns *two* embedding tables, W and $\tilde{W}$. With a symmetric context they are equivalent up to random initialisation, so the default output is their sum $\mathbf{w}_i + \tilde{\mathbf{w}}_i$, which acts as a small ensemble and usually scores a little better.

3.4 Gradients

Write $\delta_{ij} = \mathbf{w}_i^\top \tilde{\mathbf{w}}_j + b_i + \tilde{b}_j - \log X_{ij}$ (the code calls this `diff`) and $g_{ij} = f(X_{ij})\,\delta_{ij}$ (the code's `fdiff`). For a single term of $\frac{1}{2}J$:

$$\nabla_{\mathbf{w}_i} = g_{ij}\,\tilde{\mathbf{w}}_j, \qquad \nabla_{\tilde{\mathbf{w}}_j} = g_{ij}\,\mathbf{w}_i, \qquad \partial_{b_i} = \partial_{\tilde{b}_j} = g_{ij}. \tag{5}$$

The implementation reports the cost as $\frac{1}{2} f \delta^2$, so the factor 2 drops out.

3.5 Relationship to other models

The paper shows that skip-gram's objective, summed over the corpus, can be written as $-\sum_{i,j} X_{ij} \log Q_{ij}$, a weighted cross-entropy between the empirical P and the softmax model Q. GloVe replaces (a) cross-entropy with squared error on log counts, which avoids the softmax normalisation, and (b) the implicit weight X_{ij} with the bounded weight $f(X_{ij})$. In this sense GloVe is a weighted, log-count matrix factorisation that is trained by stochastic gradient descent (SGD).

Part II
The Implementation

4 Repository map

Path	Lines	Role
`src/common.{h,c}`	217	Shared types (`CREC`, the co-occurrence record; `HASHREC`, the hash-table record), the hash function, the tokenizer `get_word()`, command-line parsing `find_arg()`.
`src/vocab_count.c`	172	Stage 1: unigram counts → `vocab.txt`.
`src/cooccur.c`	445	Stage 2: windowed co-occurrence counting with a hybrid dense/sparse store and external merge sort → `cooccurrence.bin`.
`src/shuffle.c`	210	Stage 3: out-of-core shuffle → `cooccurrence.shuf.bin`.
`src/glove.c`	559	Stage 4: multi-threaded AdaGrad training → `vectors.{txt,bin}`.
`Makefile`	48	Builds four executables in `build/`.
`demo.sh`	57	End-to-end run on `text8` plus evaluation.
`randomization.test.sh`	–	Determinism tests (seeds, checkpoint/resume).
`eval/python/`	–	Analogy evaluation, nearest neighbours, interactive analogies.
`eval/{matlab,octave}/`	–	Same evaluation for MATLAB/Octave.
`eval/question-data/`	–	The Google analogy test set (14 files).

The system is **four independent Unix filters** connected by files, with no shared state beyond those files. Each tool reads standard input (`stdin`) and/or named files, writes its result to standard output (`stdout`) or named files, and writes logs to standard error (`stderr`). This keeps each stage simple, lets you re-run any stage alone (for example, retrain with a new d without recounting), and lets every stage except the trainer work out of core.

5 The pipeline at a glance

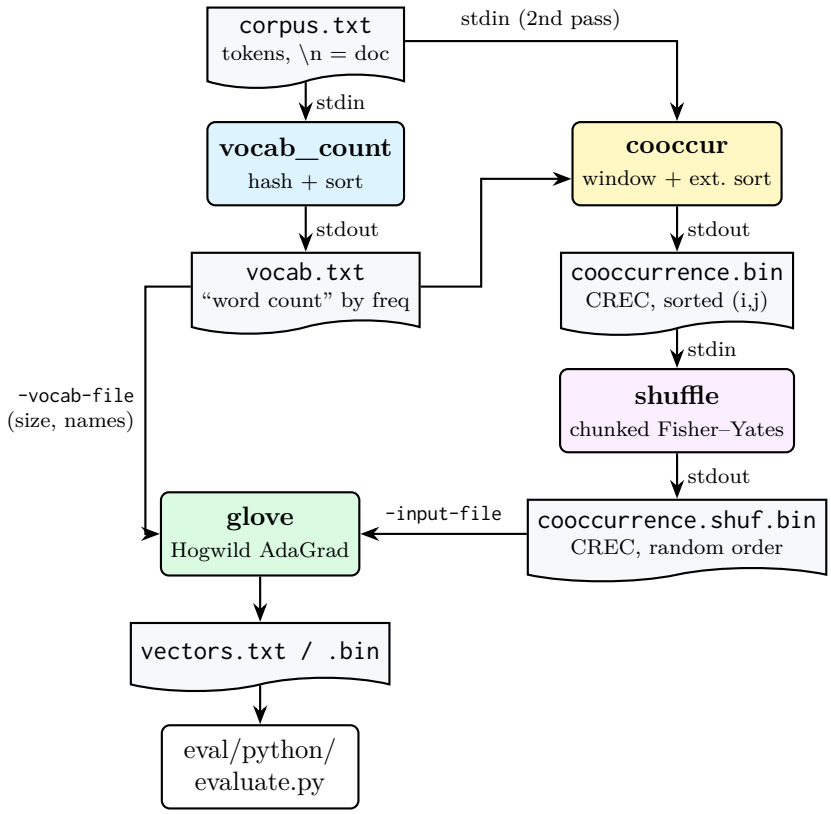


Figure 2: Data flow. The corpus is read twice, once for unigrams and once for co-occurrences. Everything after `cooccur` works on a compact binary list of non-zero cells.

Listing 1: The canonical invocation (from `demo.sh`).

```
build/vocab_count -min-count 5 -verbose 2                 < text8 > vocab.txt
build/cooccur -memory 4.0 -vocab-file vocab.txt -window-size 15 < text8 > cooccurrence.
    bin
build/shuffle -memory 4.0                        < cooccurrence.bin > cooccurrence.shuf.
    bin
build/glove -save-file vectors -threads 8 -input-file cooccurrence.shuf.bin \
            -x-max 10 -iter 15 -vector-size 50 -binary 2 -vocab-file vocab.txt
```

6 Shared foundations: `common.c`

6.1 The universal record: `CREC`

```
typedef double real;
typedef struct cooccur_rec { int word1; int word2; real val; } CREC;   // 4+4+8 = 16
    bytes
```

Every binary file between stages 2, 3 and 4 is a headerless array of `CREC`s written with `fwrite()`. Consequences:

- File size = $16 \times \mathrm{nnz}(X)$ bytes, and the record count is `filesize/16` (this is how `train_glove()` sizes its work).
- Word ids are **1-based frequency ranks**: id 1 is the most frequent word, which is line 1 of `vocab.txt`.
- The format is native-endian and native-layout, so it is not portable across architectures with different endianness or padding.

6.2 Tokenisation contract: `get_word()`

The tools do no linguistic processing. The corpus must already be tokenised and, if you want it, lowercased (`Training_README.md` uses the Stanford Penn Treebank (PTB) tokenizer with `-lowerCase`). `get_word()` follows these rules:

- Tokens are separated by spaces, tabs, or newlines. `\r` is ignored.
- **A newline marks a document boundary**. The function returns 1 for a newline or end of file (EOF) on its own, and `cooccur` then resets its window, so context never crosses lines.
- Tokens longer than 999 bytes are truncated, with care not to split a multi-byte UTF-8 (Unicode Transformation Format, 8-bit) sequence.

6.3 Hash table

A fixed array of `TSIZE` $= 2^{20}$ buckets (`TSIZE` = table size) with separate chaining, a cheap shift-xor string hash (`bitwisehash()`, by Hugh Williams), and **move-to-front on access**. Word frequencies follow Zipf's law, so a few words account for most lookups. Moving a found record to the head of its chain makes those lookups close to $O(1)$ even when chains get long for large vocabularies.

7 Stage 1 — `vocab_count`

Where in the code

`src/vocab_count.c`: `hashinsert()`, `get_counts()`, `CompareVocab()`, `CompareVocabTie()`.

Algorithm.

1. Stream tokens from `stdin` into the hash table, incrementing counts. The literal token `<unk>` is rejected because it is reserved for the output.
2. Copy all hash records into an array.
3. If `-max-vocab` N is set and exceeded, first sort by count *without* a tie-break. Words with equal counts then end up in a pseudo-random order, so truncation at N does not favour the start of the alphabet.
4. Sort the first N by (count desc, word asc).
5. Print `word count` lines, stopping at the first count below `-min-count`.

Output contract. Line r of `vocab.txt` is the word with id r. Every later stage relies on this: `cooccur` assigns ids by line number, and `glove` counts lines to get V and re-reads the file to name output rows. **Never edit `vocab.txt` between stages.**

8 Stage 2 — `cooccur`

Where in the code

`src/cooccur.c`: `get_cooccurrence()` (counting), `write_chunk()`, `merge_files()` (k-way merge), `insert()`/`delete()` (binary heap), and the memory heuristic in `main()`.

This is the most heavily engineered stage. For a corpus of billions of tokens, X has V^2 cells ($\sim 10^{11}$ for $V = 400$k) but only billions of non-zeros, and their distribution is very skewed.

8.1 Windowed counting

The tool keeps a circular buffer `history[window_size]` of the ids of the previous in-vocabulary tokens on the current line. For each new in-vocabulary target w_2 at position j, it visits each left-context word w_1 at position $k \in [j-W, j-1]$ and adds

$$\text{increment} = \begin{cases} 1/(j-k) & \texttt{-distance-weighting}\ 1\ \text{(default)} \\ 1 & \text{otherwise} \end{cases}$$

to $X_{w_1 w_2}$. With `-symmetric` 1 (the default) it also adds the same amount to $X_{w_2 w_1}$. That is how right context is counted without ever looking ahead.

Gotcha

Out-of-vocabulary tokens are *skipped entirely*: they are not added to `history` and they do not advance the position j. Distances are therefore measured in *in-vocabulary* tokens, and a high `-min-count` makes the effective window span more raw text.

Worked example. Corpus line `the cat sat on the mat`, window $W = 2$, symmetric, distance-weighted. Vocabulary ids: the=1, then ties sorted alphabetically: cat=2, mat=3, on=4, sat=5.

j	target	left context (distance)	increments (both directions)
1	cat	the (1)	$X_{\text{the,cat}}$ += 1
2	sat	cat (1), the (2)	$X_{\text{cat,sat}}$ += 1, $X_{\text{the,sat}}$ += $\frac{1}{2}$
3	on	sat (1), cat (2)	$X_{\text{sat,on}}$ += 1, $X_{\text{cat,on}}$ += $\frac{1}{2}$
4	the	on (1), sat (2)	$X_{\text{on,the}}$ += 1, $X_{\text{sat,the}}$ += $\frac{1}{2}$
5	mat	the (1), on (2)	$X_{\text{the,mat}}$ += 1, $X_{\text{on,mat}}$ += $\frac{1}{2}$

Result: $X_{\text{the,sat}} = 1.0$ (two half-weight hits), $X_{\text{the,cat}} = X_{\text{cat,sat}} = X_{\text{sat,on}} = X_{\text{on,the}} = X_{\text{the,mat}} = 1$, $X_{\text{cat,on}} = X_{\text{on,mat}} = 0.5$, and the transposes have the same values. Sixteen non-zero `CRECs` are emitted.

8.2 Hybrid dense/sparse storage

The key design decision follows from Zipf's law: if words are ranked by frequency, the count of the word with rank r is roughly proportional to $1/r$, so the expected co-occurrence count of (a, b) falls off roughly like $1/(ab)$. Pairs with a **small rank product** are therefore dense and hot, and pairs with a large product are sparse.

Dense part (`bigram_table`). A ragged 2-D array. Row a has $\min(\lfloor P/a \rfloor, V)$ columns. A prefix-sum table `lookup` gives the start of each row:

```
lookup[0] = 1;
for (a = 1; a <= vocab_size; a++)
    lookup[a] = lookup[a-1] + min(max_product / a, vocab_size);
// cell (w1,w2), both 1-based:
bigram_table[lookup[w1-1] + w2 - 2] += 1.0 / distance;
```

Updates are $O(1)$ in-place additions of `doubles`, and the hottest pairs, which make up most traffic, never touch the slow path.

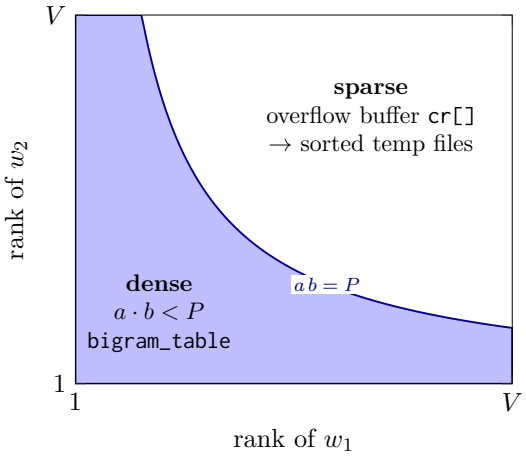


Figure 3: Partition of the $V \times V$ matrix by rank product. The region under the hyperbola $a\,b < P$ (`max_product`) is stored in a ragged dense array and updated in place. Everything else is appended to an in-memory overflow list.

Sparse part (overflow buffer). Pairs with $w_1 \geq P/w_2$ are appended as raw `CRECs` to `cr[]`, with duplicates allowed. When the buffer is nearly full (it keeps room for $2W$ more entries, the most one token can add), the tool:

1. `qsort()`s it by (w_1, w_2),
2. collapses adjacent duplicates while writing (`write_chunk()`),
3. writes it to `overflow_NNNN.bin` (NNNN ≥ 1).

At EOF the last buffer is flushed the same way, and the whole dense table is written in row-major order (already sorted, zeros skipped) as `overflow_0000.bin`.

8.3 External k-way merge

Now there are F files, each sorted by (w_1, w_2), and the same pair may appear in several of them. `merge_files()` opens all of them and keeps a **binary min-heap** (`CRECID` = `CREC` + source file id) holding one head record per file. It repeatedly pops the minimum, adds it to a running "current" record if the key matches, or writes the current record to `stdout` and starts a new one if it does not, then refills from the popped record's file. The output is **globally sorted with unique keys**. Temp files are deleted afterwards.

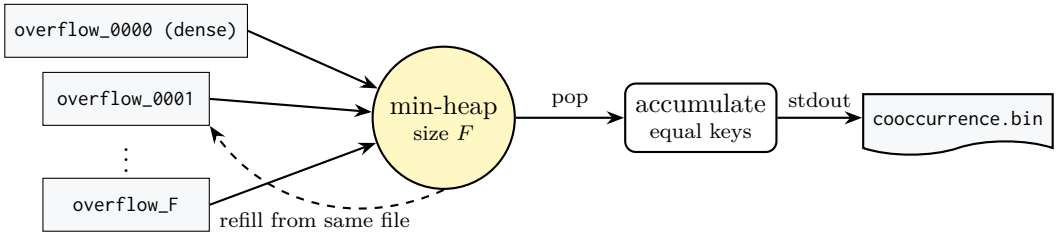


Figure 4: Final phase of `cooccur`: an F-way merge that also sums duplicates.

8.4 Choosing `max_product` from `-memory`

`main()` turns a memory budget M in gigabytes (GB) into the two buffer sizes:

$$R = \frac{0.85\,M \cdot 2^{30}}{\texttt{sizeof(CREC)}}, \qquad \texttt{overflow_length} = R/6,$$

and solves for $P = \texttt{max_product}$ by fixed-point iteration:

$$P\,(\ln P + 0.1544313298) = R.$$

Where does the constant come from? The number of dense cells is about the number of integer pairs with $a\,b \le P$, which is $\sum_{k \le P} d(k)$, the divisor summatory function. Dirichlet's estimate gives $\sum_{k \le P} d(k) \approx P \ln P + (2\gamma - 1)P$, and $2\gamma - 1 = 0.15443\ldots$ (γ = Euler–Mascheroni). The heuristic therefore sizes the hyperbolic region of Figure 3 to use about 85% of the budget, leaving about 1/6 for the overflow buffer. Because cells are divided by `sizeof(CREC)`=16 while dense cells are 8-byte `doubles`, the estimate is conservative. Use `-max-product` and `-overflow-length` to override it.

Key insight

If you give `cooccur` more memory, more pairs go to the fast dense path, fewer temp files are written, and the merge has a smaller fan-in. The output is identical either way. Only speed and disk use change.

9 Stage 3 — `shuffle`

Where in the code

`src/shuffle.c`: `shuffle_by_chunks()`, `shuffle_merge()`, `shuffle()` (Fisher–Yates), `rand_long()`.

Why shuffle at all? `cooccurrence.bin` is sorted by w_1. Training on it in that order would apply thousands of consecutive updates to the same row $\mathbf{w}_1$, then $\mathbf{w}_2$, and so on. That is very biased SGD. It also defeats Hogwild parallelism (Section 10.3), because neighbouring records, and records at the same offset in different threads' slices, would share rows.

Algorithm (two-pass, out-of-core).

1. **Chunk shuffle.** Read up to `array_size` records ($0.95\,M \cdot 2^{30}/16$ by default), Fisher–Yates shuffle them in memory, and write `temp_shuffle_NNNN.bin`. Repeat until EOF.
2. **Interleaving merge.** Repeatedly read about `array_size`$/F$ records from *each* temp file into one buffer, shuffle the buffer, and write it to `stdout`.

This is not a perfectly uniform permutation (the code says "good enough"), but every output window mixes records from every part of the input. `rand_long()` combines two `rand()` calls and uses rejection sampling to avoid modulo bias. `-seed` makes the result reproducible; without it the seed is `time(0)`.

If the input fits in one chunk ($F = 1$), the result is a single uniform Fisher–Yates permutation followed by a second shuffle.

10 Stage 4 — `glove`, the trainer

Where in the code

`src/glove.c`: `initialize_parameters()`, `glove_thread()` (the inner loop), `train_glove()` (epoch driver), `save_params()` (output), `main()` (command-line parsing).

10.1 Parameter memory layout

All parameters live in one contiguous, 128-byte-aligned array `W` of $2V(d+1)$ doubles. A parallel array `gradsq` of the same shape holds the AdaGrad accumulators.

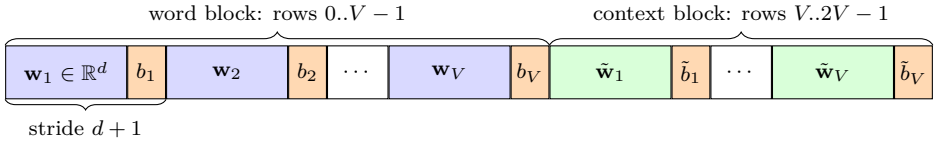


Figure 5: Layout of `W` (and `gradsq`). For a record (i, j) the trainer computes `l1`$= (i-1)(d+1)$ and `l2`$= (j-1+V)(d+1)$. The bias sits at offset d inside each row, so a vector and its bias share cache lines.

Initialisation. $W_k \sim \mathcal{U}(-0.5, 0.5)/d$ for every entry, biases included, and $\texttt{gradsq}_k = 1$. Alternatively, both arrays can be loaded from binary checkpoints (`-load-init-param`, `-load-init-gradsq`).

10.2 The inner loop

Listing 2: Core of `glove_thread()`, lightly condensed.

```
for each CREC cr in my slice of the file:
    l1 = (cr.word1 - 1) * (d+1);                  // word row
    l2 = (cr.word2 - 1 + V) * (d+1);              // context row
    diff  = dot(W+l1, W+l2, d) + W[l1+d] + W[l2+d] - log(cr.val);
    fdiff = (cr.val > x_max) ? diff : pow(cr.val/x_max, alpha) * diff;
    if (isnan/isinf(diff or fdiff)) continue;
    cost[id] += 0.5 * fdiff * diff;

    for b in 0..d-1:                               // vectors: AdaGrad + clipping
        t1 = clip(fdiff * W[l2+b], +-grad_clip) * eta;   // grad wrt w_i
        t2 = clip(fdiff * W[l1+b], +-grad_clip) * eta;   // grad wrt w~_j
        u1[b] = t1 / sqrt(gradsq[l1+b]);   gradsq[l1+b] += t1*t1;
        u2[b] = t2 / sqrt(gradsq[l2+b]);   gradsq[l2+b] += t2*t2;
    if (sum(u1), sum(u2) finite) { W[l1+b] -= u1[b]; W[l2+b] -= u2[b]; }

    W[l1+d] -= fdiff / sqrt(gradsq[l1+d]);         // biases: AdaGrad, no eta
    W[l2+d] -= fdiff / sqrt(gradsq[l2+d]);
    gradsq[l1+d] += fdiff*fdiff;  gradsq[l2+d] += fdiff*fdiff;
```

Written as math, for vector component k:

$$u = \eta \cdot \mathrm{clip}(g_k), \qquad w_k \leftarrow w_k - \frac{u}{\sqrt{G_k}}, \qquad G_k \leftarrow G_k + u^2, \qquad G_k^{(0)} = 1.$$

This is AdaGrad with three implementation details worth knowing:

1. The accumulator stores the *scaled* gradient $(\eta g)^2$, and G starts at 1, so the first step has size exactly $\eta\, g$. This is what the comment "So initial value of eta is equal to initial learning rate" means.
2. The divisor is the accumulator *before* this step's gradient is added.
3. Both u_1 and u_2 are computed from the *old* $\mathbf{w}_i$ and $\tilde{\mathbf{w}}_j$ before either is written, so the update is a true simultaneous gradient step.

Gotcha

Bias updates do not use η and are not clipped: the step is $g/\sqrt{G}$ with G accumulating g^2. Changing `-eta` therefore rescales the vector learning rate but not the bias learning rate. Also, when the NaN (not-a-number) guard rejects a vector update, `gradsq` has already been incremented.

10.3 Hogwild parallelism

Each epoch, `train_glove()`:

1. splits the N records into T contiguous slices of $\lfloor N/T \rfloor$ (the last thread also takes the remainder);
2. starts T POSIX threads (pthreads). Each one opens its *own* `FILE*`, seeks with `fseeko()` to id $\cdot \lfloor N/T \rfloor \cdot 16$, and streams its slice;
3. joins all threads and prints the mean cost $\sum_t \texttt{cost}[t]/N$.

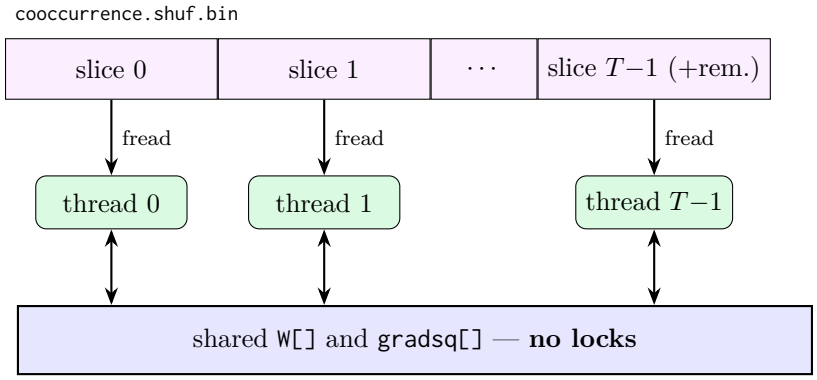


Figure 6: One epoch. Threads read disjoint slices of the shuffled file and update shared parameters without synchronisation (Hogwild!, Recht et al. 2011).

Why lock-free is acceptable. Each update touches only $2(d+1)$ entries in each of `W` and `gradsq`, out of $2V(d+1)$. After shuffling, two threads rarely touch the same row at the same moment, and when they do, the race costs at most a lost or stale update. SGD tolerates that noise. The exception is the most frequent words (ids near 1), which appear in many records and are contended most. They also receive the most updates, so they converge anyway.

Gotcha

Multithreaded training is **not bit-reproducible**, even with `-seed`, because thread interleaving is nondeterministic. `randomization.test.sh` uses `-threads 1` for this reason.

10.4 Outputs

`-binary`	Files written
0 (default)	`vectors.txt` only
1	`vectors.bin` only: the raw `W` array, $2V(d+1)$ doubles, layout as in Figure 5. Reshape to $(2V, d+1)$ to read it.
2	both

`-model`	Content of each line of `vectors.txt` (`word` v_1 `. . .`)
0	$[\mathbf{w}_i, b_i, \tilde{\mathbf{w}}_i, \tilde{b}_i]$ — everything, $2(d+1)$ numbers
1	$\mathbf{w}_i$ only
2 (default)	$\mathbf{w}_i + \tilde{\mathbf{w}}_i$
3	$[\mathbf{w}_i, \tilde{\mathbf{w}}_i]$ — documented in `-help`, see §15

The <unk> vector. With `use_unk_vec`=1 (hard-coded), the text output gains a final row `<unk>` whose vector is the mean of the $\min(100, V)$ *rarest* words' parameters. Rare words are the best available proxy for an unseen word. This is why `<unk>` is banned from the corpus.

Header. `-write-header` 1 prepends `"V d"`. This makes the file readable as word2vec text format (for example by gensim), but note that V excludes the extra `<unk>` row.

Checkpointing. `-checkpoint-every` k writes `vectors.NNN.bin` and/or `vectors.NNN.txt` every k epochs, and `-save-init-param` writes epoch `000`. Combined with `-gradsq-file` and the `-load-init-param`/`-load-init-gradsq` flags, this gives **exact resume**. `randomization.test.sh` checks that "2 epochs, then resume for 4" is byte-identical to "6 epochs straight" when run single-threaded on the same shuffled file.

Part III
Operating GloVe

11 Hyper-parameter reference

Tool	Flag	Default	Effect / guidance
vocab_count	`-min-count`	1	Drop rare words. 5 for text8, 20 for Wikipedia+Gigaword.
	`-max-vocab`	0 (none)	Hard cap on V (Dolma run used 1.2M).
cooccur	`-window-size`	15	Context radius W. Smaller windows emphasise syntax, larger ones topical semantics. 10 in the 2024 runs.
	`-symmetric`	1	0 = left context only.
	`-distance-weighting`	1	$1/d$ weighting of pairs.
	`-memory`	3 (help says 4)	Soft budget in GB (see §8.4).
	`-max-product`, `-overflow-length`	auto	Manual overrides.
	`-overflow-file`	overflow	Temp file prefix. Put it on fast disk.
shuffle	`-memory`	2 (help says 4)	Chunk size budget.
	`-seed`	time	Reproducible shuffles.
	`-temp-file`	temp_shuffle	Temp file prefix.
glove	`-vector-size`	50	d.
	`-iter`	25	Epochs. 50–100 for large corpora.
	`-eta`	0.05	Initial AdaGrad rate (vectors only).
	`-alpha`	0.75	Exponent in f.
	`-x-max`	100	Cutoff in f. Use about 10 for small corpora.
	`-grad-clip`	100	Per-component gradient clip.
	`-threads`	8	Hogwild workers.
	`-binary`, `-model`	0, 2	Output format (see §10.4).
	`-seed`	time	Parameter initialisation.

12 Resource sizing

- **Trainer RAM** (random-access memory) $\approx 2 \times 2V(d+1) \times 8$ bytes (`W` + `gradsq`). Example: $V = 400$k, $d = 300$: $2 \times 2 \cdot 400{,}000 \cdot 301 \cdot 8 \approx 3.9$ GB.
- **Co-occurrence file** $= 16 \times \mathrm{nnz}(X)$ bytes. The shuffled copy is the same size, and `shuffle` needs a third copy of temp space while it runs, so plan disk for about 3×.
- **Epoch cost** $= O(\mathrm{nnz}(X) \cdot d)$, independent of corpus length. The paper argues that, under a power-law model of counts, $\mathrm{nnz}(X)$ grows sub-linearly in corpus size $|C|$ (about $|C|^{0.8}$ for their data). This is the main efficiency argument against window-scanning methods.
- **I/O (input/output) pattern**: every stage streams sequentially. `glove` re-reads the whole shuffled file every epoch, T streams in parallel, so page-cache or solid-state drive (SSD) throughput matters.

13 Evaluation

`eval/python/evaluate.py` runs the Google analogy benchmark:

1. Load `vocab.txt` and `vectors.txt` (skipping `<unk>`), then L2-normalise every row.
2. For each question $a : b :: c :?$, compute $\mathbf{q} = \hat{\mathbf{w}}_b - \hat{\mathbf{w}}_a + \hat{\mathbf{w}}_c$ and predict $\arg\max_x \hat{\mathbf{w}}_x^\top \mathbf{q}$, excluding a, b, c. Questions are processed in batches of 100 as one matrix product.
3. Report top-1 accuracy per file and aggregated: the first five files are *semantic* (capitals, currency, family, ...) and the other nine are *syntactic* (plurals, tenses, ...). Questions with out-of-vocabulary words are skipped, and "Questions seen/total" reports coverage.

`distance.py` gives interactive nearest neighbours and `word_analogy.py` gives interactive analogies. The MATLAB and Octave directories implement the same benchmark.

14 Reproducibility

Stage	Source of randomness	Control
vocab_count	none; ties are broken alphabetically (`-max-vocab` truncation depends on `qsort()` order)	—
cooccur	none	—
shuffle	`rand()`	`-seed`
glove	init `rand()`; thread scheduling	`-seed` + `-threads 1`

15 Known quirks found while reading the code

These are observations from the current source, not documented behaviour.

1. **`-model 3` is unreachable.** `main()` in `glove.c` runs `if (model != 0 && model != 1) model = 2;`, so a requested 3 is silently changed to 2, even though `save_params()` has a branch for 3 and `-help` documents it.
2. **Bias learning rate ignores `-eta`** and is not clipped (see Listing 2).
3. **Default/help mismatches.** `cooccur`'s `memory_limit` defaults to 3 GB and `shuffle`'s to 2 GB, while both help texts say 4.0. The `-memory` flag overrides both.
4. **Help invocation differs by tool.** `cooccur` and `glove` print help when given no arguments, while `vocab_count` and `shuffle` need `-h`, because they read `stdin` by default.
5. **`-Ofast` breaks NaN guards.** As the `Makefile` warns, `-ffast-math` assumes finite values, so `isnan()`/`isinf()` checks are optimised away. The default build uses `-O3`.
6. **`calloc(num, sizeof(FILE))`** in `merge_files()` and `shuffle_merge()` allocates `sizeof(FILE)` instead of `sizeof(FILE*)` per slot. This over-allocates but is otherwise harmless.

16 Mental model: one-paragraph summary

GloVe in one paragraph

Count how often every pair of words appears near each other, discounting by distance and never crossing a newline. Do this once, using a dense array for the Zipf-head pairs and sorted spill files for the long tail, then merge everything into one sorted list of non-zero (i, j, X_{ij}) records. Shuffle that list. Then, for 15–100 passes, let several threads stream disjoint parts of it and, without locking, nudge $\mathbf{w}_i^\top \tilde{\mathbf{w}}_j + b_i + \tilde{b}_j$ towards $\log X_{ij}$, weighting each nudge by $f(X_{ij})$ and scaling it per coordinate with AdaGrad. At the end, write $\mathbf{w}_i + \tilde{\mathbf{w}}_i$ for each word, plus an `<unk>` vector averaged from the rarest words.

References

[1] J. Pennington, R. Socher, C. D. Manning. *GloVe: Global Vectors for Word Representation*. Conference on Empirical Methods in Natural Language Processing (EMNLP) 2014. https://nlp.stanford.edu/pubs/glove.pdf

[2] T. Mikolov et al. *Distributed Representations of Words and Phrases and their Compositionality*. Conference on Neural Information Processing Systems (NeurIPS) 2013.

[3] J. Duchi, E. Hazan, Y. Singer. *Adaptive Subgradient Methods for Online Learning and Stochastic Optimization* (AdaGrad). Journal of Machine Learning Research (JMLR) 2011.

[4] B. Recht, C. Re, S. Wright, F. Niu. *Hogwild!: A Lock-Free Approach to Parallelizing Stochastic Gradient Descent*. NeurIPS 2011.

[5] Project page and pre-trained vectors: https://nlp.stanford.edu/projects/glove/